The Cost of Starting Over:

Predicting When LPA* Beats Replanning with A* in Robot Navigation

CLAUDIUS MARCO ANDREW

Comp3190 W26 - John Anderson

Abstract

This paper studies repeated replanning in dynamic robot navigation by comparing two search strategies (1) rerunning **A* from scratch** after each map update and (2) **repairing** an existing plan using **Lifelong Planning A* (LPA*).** The report reviews foundational and recent research on both heuristic and incremental search to explain why replanning is important, what LPA* changes compared to A*, and what trade offs appear when the algorithms face environmental changes over time. At its core, the paper aims to identify a practical threshold for when restarting with A* is sufficient and when repairing the search with LPA* becomes more effective.

It then presents a small Rviz2 based experiment using grid maps with dynamic obstacles to compare the algorithms under different map sizes and levels of environmental change. The performance of the algorithms is evaluated using planning time, path length, and time to the goal, with the overall aim of identifying a practical guideline for when restart based replanning is sufficient and when incremental repair becomes more efficient.

1. Introduction, Problem Setup and Background

This section inspired the topic on why repeated replanning is a meaningful problem especially in robot navigation and introduces the key AI ideas needed to compare "restart" planning (rerunning A*) against "repair" planning (incrementally updating a prior search, here using Lifelong Planning A*, LPA*). Utilizing papers on traditional incremental search work as well as recent navigation focused "when to replan" study, the section explores the question: under what conditions (change frequency and change size) does incremental repair become preferable to restarting from scratch? (Koenig & Likhachev, 2004; Koenig et al., 2004; Honda et al., 2023).

1.1 Dynamic navigation: why replanning matters

In real navigation tasks, robots rarely operate in a perfectly static world. Factors such as new obstacles appearing, costs in the environment changing, etc. In this setting, path planning is not a one-time computation, robots often need to recompute or update its route repeatedly as conditions change. Replanning matters because the

robot's ability to respond quickly affects whether it can keep moving safely and efficiently rather than stopping for long planning cycles.

A framing of this problem appears in Koenig and Likhachev's *Fast Replanning for Navigation in Unknown Terrain*. They considered mobile robots moving through environments that are not fully known ahead of time, so the robot must be able to quickly revise its route as it learns more about the terrain (Koenig & Likhachev, 2004, Abstract, p. 354). Their approach highlights that replanning is a repeated cycle where the robot first plans the shortest path to the goal by assuming that any area it has not seen yet is open, then updates its map when it detects obstacles, and plans again. This process continues until the robot either reaches the goal or determines that no traversable path exists (Koenig & Likhachev, 2004, Section I, p. 354).

Likewise, Honda et al.'s *When to Replan?* proposed a similar approach in which a global planner generates a reference path using a pre-built map, while a local planner produces a possible path that follows the main reference path while avoiding perceived obstacles (Honda et al., 2023, Abstract, p. 1). They argue that, when obstacles are unforeseen or dynamic and therefore not represented in the pre-built map, the question of when to replan the global reference path becomes crucial. They also noted that the rate in which we replan can cause negative behaviors in which when replanning too often or too rarely can lead to inefficient travel (e.g., oscillating behavior) or even the robot becoming stuck (Honda et al., 2023, p. 1).

This raises the question that if replanning is frequent, then repeatedly restarting a planner from scratch may consume substantial computation time, if replanning is rare, a simpler restart strategy might be adequate. This report treats replanning as a threshold question "for what levels of change frequency and change size does it become more efficient to repair an existing plan (LPA*) rather than restart planning (A*)?"

1.2 Heuristic search: what it is and how it helps

Heuristic search is a family of methods for finding good (often shortest) paths by using an informed estimate called a heuristic. Koenig et al. states that the heuristic search is a central tool for shortest path planning. In their overview, they describe heuristic search methods as widely used in AI because they use "approximations of the goal distances" to focus the search and can find shortest paths faster than uninformed methods (Koenig et al., 2004, p. 2).

1.3 What LPA* changes compared to A*

Koenig et al. made an important differentiation by stating that LPA* is more accurately a plan reuse method than a true replanning method (2004, p. 2). In the abstract, they state that LPA* "repeatedly finds shortest paths" from a fixed start to a fixed goal while edge costs change or vertices are added or deleted (Koenig et al., 2004, Abstract, p. 1). This matters because LPA* does not solve each new problem from scratch. Instead, it updates and reuses useful results from earlier searches after changes occur. That reuse is the key reason it matters for this report's comparison with A*. A key finding is that LPA*'s first search is equivalent to that of A* but in later searches can be faster because it keeps parts of the previous search results that are still valid after an update (Koenig et al., 2004, Abstract, p. 1). Koenig et al. argue that this is especially useful in systems where planning happens repeatedly and changes are usually small, since recomputing everything each time can be wasteful (2004, p. 2). For that reason, this report treats LPA* as a repair-based approach while still keeping the authors' terminology distinction in mind.

Likhachev's D* Lite paper additionally reinforces the key intuition behind repair planning. Replanning can be faster than planning from scratch when the algorithm modifies previous search results locally. In their paper (discussing Stentz's D*), they state that D* can replan faster than planning from scratch because it modifies prior search results locally, and they introduce D* Lite as an alternative that is "at least as efficient" in their analysis (Koenig & Likhachev, 2004, Abstract, p. 354).

2. Exploring Methods: Restart vs Repair Planning

This section describes the two planning approaches compared in this report (1) restart replanning (rerunning A* from scratch when the map changes) and (2) repair replanning (incrementally updating the prior search using Lifelong Planning A*, LPA*). The goal is not to claim that one planner is superior, but to clarify their differences to support a threshold style guideline for determining when changes in frequency and map size make "repair" preferable to "restart" in dynamic navigation or vice versa.

2.1 Baseline: A* method exploration and restart replanning

A* is a heuristic search method commonly used as a baseline for shortest path planning. In a dynamic setting, a straightforward way to respond to map updates is to simply do restart replanning. Whenever new obstacles or changed costs invalidate the current plan, the system reruns A* on the updated map to compute a new shortest path.

Koenig et al. describe key operational features of A* that motivate why it is a natural baseline. In their comparison discussion, they note that A* maintains an OPEN list (a priority queue) and a CLOSED list, and that the OPEN list allows A* to expand a fringe vertex with the smallest f-value while the CLOSED list prevents re-expansion of vertices (Koenig et al., 2004, PDF p. 12). Because A* is a clear and widely understood way to compute a shortest path from scratch, it serves as a simple reference point for comparison.

A* is also a practical reference point in grid pathfinding. In Sturtevant's *Benchmarks for Grid-Based Pathfinding*, A* is treated as a standard algorithm whose effort can be measured through nodes expanded, it then compares that with heuristic quality (how good the heuristic is at guiding A* toward the goal) across selected problem groups. (Sturtevant, 2012, Section E, PDF p. 4).

2.2 Proposed: LPA* method exploration and repair replanning

Koenig, Likhachev, and Furcy define LPA* explicitly as an incremental version of A* for repeated shortest-path computation under graph changes, and they emphasize two points that matter for this report. First, LPA*

“repeatedly finds shortest paths” while edge costs change or vertices are added/deleted. Second, its first search is the same as a tie-breaking version of A*, while later searches can be faster because LPA* reuses parts of the previous search tree that remain identical (Koenig et al., 2004, Abstract, PDF p. 1).

The core mechanism of LPA* is its internal state and consistency checks. In the algorithm description, LPA* maintains (i) a g-value estimate for each vertex and (ii) a second estimate called the rhs-value, described as a one-step lookahead value derived from g-values (Koenig et al., 2004, Section 4.1, PDF p. 11). The paper states that the initial search computes g-values in exactly the same order as A*, and that LPA* then carries the g-values forward from search to search (Koenig et al., 2004, Section 4.1, PDF p. 11). This “carry forward” detail is central to the repair behavior, which makes reuse possible because the algorithm does not throw away its prior distance estimates.

At the algorithm level, Koenig et al. shows LPA* in pseudocode and describe its repeated planning loop. Initialize() sets initial g-values to infinity, sets rhs-values (with the start having rhs = 0), and inserts the start into a priority queue with a computed key. Initializing the g-value to infinity represents that no finite path to those nodes is known yet. The algorithm then repeatedly expands the node with the smallest key, using that key to decide which node is currently the most promising for finding the shortest path.

They state that this initialization sets the first call to ComputeShortestPath() performs exactly an A* search (under suitable A* tie-breaking), expanding the same vertices in the same order (Koenig et al., 2004, Section 4.2, PDF p. 13). After that, the algorithm “waits for changes in edge costs,” calls UpdateVertex() to update rhs-values/keys for affected vertices and then calls ComputeShortestPath() again to repair the shortest path (Koenig et al., 2004, Figure 6, pp. 11–13). This will essentially be the template for repair replanning used in this report, where changes do not trigger a full restart, but instead trigger localized updates in which only the affected areas are recomputed to make the relevant vertices consistent again.

2.3 Comparing A* and LPA* in restart versus repair planning

Koenig et al. provide several formal similarities and differences that clarify why comparing these two approaches is reasonable. They state that LPA* expands vertices in a manner similar to A*, including using a priority ordering that corresponds to A*’s f-values and a tie breaking idea related to LPA*’s g-values (Koenig et al.,

2004, Section 4.2, PDF p. 13-16). Specifically, when two vertices have the same f-value, LPA gives precedence to the vertex with the smaller g-value. For this report, this offers a concrete (and measurable) trade-off where, repair planning may carry out more work for each vertex in the worst case, but when changes are small, later searches are able to expand far fewer vertices overall.

In a restart approach, rerunning A* has no mechanism to preserve and reuse the previous run's correct g-values (because the search is restarted), so it may expand on already expanded vertices even if those parts of the map did not change. In contrast, LPA*'s repair step is designed explicitly to skip expanding vertices that were already correct before the update, focusing work on the impacted region (Koenig et al., 2004, Section 5.3, p. 17).

3. Replanning Threshold Model: Guideline for Dynamic Navigation

Section 3 proposes a small, testable guideline for deciding when to restart planning with A* versus when to "repair" a prior search with Lifelong Planning A* (LPA*) after an update. Prior work motivates this choice by noting that repeated replanning is common and that restarting from scratch can be inefficient when changes are frequent while most of the graph remains unchanged (Koenig et al., 2004, p. 2; Koenig & Likhachev, 2004, p. 354). The guideline is stated as a heuristic and will be checked in the experiment described later.

3.1 Developing a simple rule for restart versus repair

In this report, the "threshold model" is a decision rule that chooses one of two responses to a map update. Either to rerun A* from scratch (restart) or update and reuse an existing search using LPA* (repair). The rule does not attempt to predict exact runtime but instead, it classifies update conditions under which restart is likely "good enough" versus conditions where reuse is likely worth.

Likhachev, and Furcy describe search as a repetitive process in many to most AI systems and note that many to even most systems "replan from scratch," solving each new path-planning problem independently. They argue this can be inefficient in large domains with frequent changes which can greatly reduce how quickly the system responds (Koenig et al., 2004, p. 2). From a robotics perspective, Likhachev similarly warns that planning from scratch can be inefficient because *"most edge costs do not change between replanning episodes,"* and they encourage replanning through modification of using previous search results (Koenig & Likhachev, 2004, p. 354).

3.2 Guidelines basis: change frequency and change size

To make the threshold rule testable, this report describes map updates using two inputs: **(i) change frequency** and **(ii) change size**. Change frequency measures how often updates occur during navigation, while change size measures how much of the map changes each time, reported as changed cells and percentage of the map affected. Together, these inputs are summarized as **disturbance level**, a practical description of how disruptive

updates are to the robot's current plan. This keeps the focus on building a realistic replanning threshold rather than claiming that one method is always better.

This choice is supported by prior work that treats replanning frequency as a practical issue. Honda et al. point out that if replanning happens too often or too rarely, navigation can become inefficient, such as repeatedly switching paths (oscillation) or even getting stuck (Honda et al., 2023, p. 1). Using grids also makes these inputs practical to measure. In a grid environment, it is straightforward to count how many cells changed and how often updates occur, and grid maps are commonly used in dynamic settings where the world or movement costs can change over time (Sturtevant et al., 2015, p. 10). Because grid representations are relatively easy to update as the environment changes, they are a reasonable experimental setting for testing how different replanning strategies respond to update frequency and update size (Sturtevant et al., 2015, p. 10).

3.3 Practical takeaway: a clear, non-formulaic guideline

The report's practical takeaway is a short rule that could possibly guide an engineering choice. The guideline here is intentionally non formulaic and is presented as a hypothesis, "when does frequent/local updates should favor repair, while rare/widespread updates should favor restart". The purpose of the experiment is to check whether that statement matches observed performance patterns.

Koenig et al. report that LPA* can have an advantage when path planning problems "change only slightly," because it can reuse parts of the previous search tree instead of rebuilding everything but at the same time, their analysis shows that LPA*'s ComputeShortestPath() has an additional overhead that may not pay off if there are only a few updates (Koenig et al., 2004, pp. 15–16).

To complement this finding another analysis in *Anytime Dynamic A* (AD*).* Likhachev et al. note that if changes in edge costs are "substantial," repairing the current solution can be computationally expensive (Likhachev et al., 2004, p. 5).

4. Experimental Setup & Results

4.1 Experimental setup

This report uses a small Rviz2-based grid experiment to compare restart replanning with A* against repair replanning with LPA* under controlled map changes. In each trial, the planner must move from a start cell to a goal cell while the map is updated during navigation. These updates block part of the current route and force the planner to generate a new path. This setup follows the project goal of comparing the two strategies under repeated map changes rather than only on a static shortest-path problem.

Three grid sizes are used: 10×10, 30×30, and 50×50. Varying the map size makes it possible to observe whether the comparison changes as the search space becomes larger. The proposal originally planned to vary both change frequency and change size, but to keep the experiment manageable these are combined into a single disturbance level variable. In the actual trials, low disturbance means smaller map changes introduced every 5 seconds, while high disturbance means larger changes introduced every 1 second. This gives a simple contrast between less frequent updates and frequent updates without requiring every possible combination of conditions. The reason for varying update frequency is that replanning is not only about how much the map changes, but also about how often the planner is forced to respond. Honda et al. argue that replanning timing is a practical issue because replanning too frequently or too infrequently can hurt navigation efficiency, sometimes cause oscillation or even get the robot stuck (Honda et al., 2023, p. 1).

4.2 Variables measured

The experiment measures **planning time, path length, and time to goal.** These were chosen because together they show both how much computation each planner uses and how well the robot actually navigates. Planning time is the main metric, since the report is trying to find when restarting with A* is cheaper than repairing with LPA*. Path length is included to check that a faster planner is not only faster because it produces a worse route. Time to goal is used as an overall outcome, since a planner that updates quickly is only useful if the robot can still reach the goal efficiently.

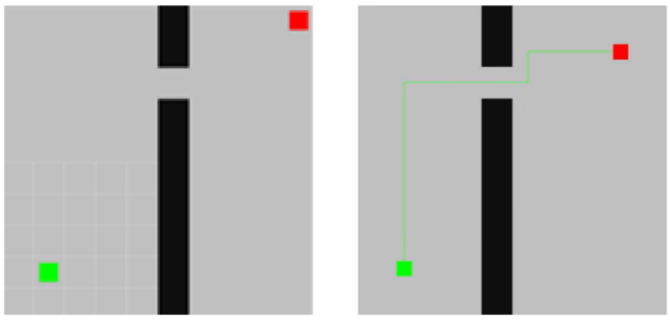


Figure 1. Example of replanning in a constrained grid layout

In both panels, the **green square** indicates the start cell, the **red square** indicates the goal cell, and the black cells indicate obstacle walls. The left panel shows the initial map configuration before the route is displayed. The right panel shows the resulting **planned path**, drawn in light green, which detours around the central obstacle and uses the available opening near the top of the map. This example highlights how obstacle placement constrains the route and forces the planner to compute an indirect path.

Low disturbance maps

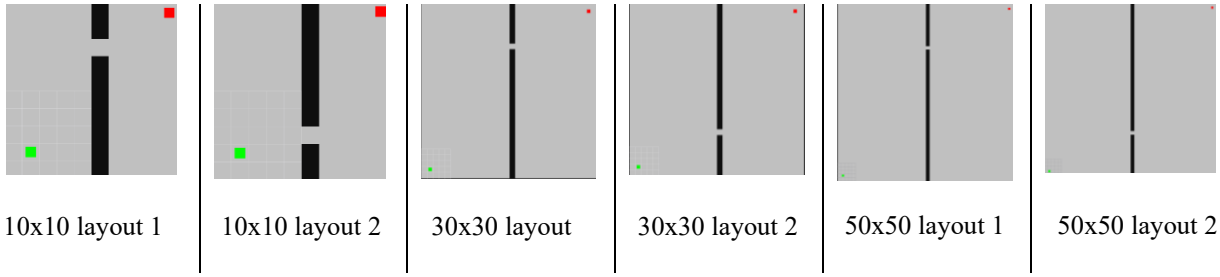
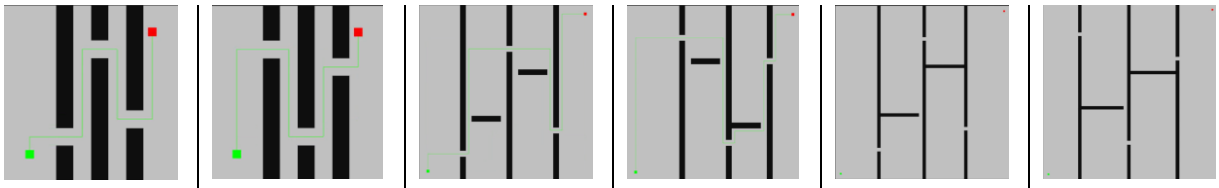


Figure 2. Low-disturbance map layouts used in the experiment

Fig2 layouts represent the milder test condition, where smaller map changes are introduced every 5 seconds. Their purpose is to test how A* and LPA* behave when updates are less frequent and less disruptive to the current route. As with the high-disturbance cases, performance is compared using planning time, path length, and time to goal.

High disturbance maps



10x10 layout 1	10x10 layout 2	30x30 layout 1	30x30 layout 2	50x50 layout 1	50x50 layout 2

Figure 3. High-disturbance map layouts used in the experiment

Fig3 layouts represent the more difficult test condition, where map changes are larger and are introduced every 1 second. Their purpose is to test how A* and LPA* behave when replanning is required frequently and when updates are disruptive enough to affect a larger portion of the current route. These cases are then compared using planning time, path length, and time to goal to evaluate both replanning cost and overall navigation performance.

These two sets of maps were included so the experiment could compare planner behavior under both less disruptive and more disruptive update conditions. Their purpose is to test how sensitive the planners are to both the size of the map change and how often replanning is required. Performance across these layouts is compared using planning time, path length, and time to goal, so that both replanning cost and overall navigation performance can be evaluated. All these layouts are tested with both A* and LPA* to allow a direct comparison of restart and repair replanning under the same disturbance conditions.

4.3 Experiment Result

Test cases	A* Planning Time	LPA* Planning Time	A* Path Length	LPA* Path Length	Notes
10x10 low disturbance	~0.06–0.13 ms	~0.08–0.27 ms, with spikes up to about 2.42 ms	15 cells	15 cells	A* stayed in a lower and tighter range of planning times, while LPA* had a slightly wider range with some bigger spikes. This suggests that on a very small grid with light changes, A* is still the more stable choice.
10x10 medium disturbance	~0.06–0.20 ms, with spikes up to about 2.43 ms	~0.05–0.28 ms with spikes up to about 0.91 ms	L1: 25 cells L2: 15 cells	L1: 25 cells L2: 15 cells	The planning time ranges were close, and LPA* showed fewer large spikes in this run. This makes the medium case more balanced than the other 10×10 tests.
10x10 high disturbance	~0.06–0.10 ms spikes around 0.20–0.24	~0.25–0.98 ms with spikes up to about 1.98 ms	L1: 23 cells L2: 25 cells	L1: 23 cells L2: 25 cells	A* stayed much lower in its planning time range, while LPA* was clearly higher and less stable. This suggests that even with stronger updates, restarting is still cheaper on a 10×10 grid.
30x30 low disturbance	~0.58–1.16 ms moderate spikes around ~1.74–2.68 ms larger spikes around ~5.97 ms ~6.12 ms ~6.59 ms	~0.13–0.34 ms after the first run spikes around: ~2.37 ms ~3.38 ms ~5.72 ms and an initial spike around ~12.79 ms	55 cells	55 cells	After the first run, LPA* usually stayed in a much lower planning time range than A*. This is the clearest case where reusing earlier search work starts to help.
30x30 high disturbance	~0.58–1.27 ms spikes around: ~6.61 ms ~7.50 ms ~10.83 ms	~7.7–12.8 ms spikes around: 22.65 ms	L1: 91 cells L2: 83 cells	L1: 91 cells L2: 83 cells	A* stayed in a much lower planning time range, while LPA* became clearly slower and showed a large spike. In this case, the cost of repairing the old search seems bigger than the benefit of reuse.
50x50, low disturbance	~1.73–5.48 ms spike around 7.12 ms	~0.24–0.64 ms spikes around: 18.5–28.5 ms initial run around 41.4 ms	L1: 95 cells L2: 95 cells	L1: 95 cells L2: 95 cells	LPA* was usually much lower in its normal planning time range after startup, although it still had some large spikes. This suggests that on the larger grid with lighter changes, LPA* can be better in typical runs.

50x50, high disturbance	~1.65–4.06 ms larger spikes about 12.76 ms	~16.5–33.9 ms after startup early run: 55.820 ms later spikes: 43.480 ms, 47.266 ms	L1: 147 cells L2: 157 cells	L1: 147 cells L2: 157 cells	A* remained much lower overall in its planning time range, while LPA* stayed much higher and showed several large spikes. One likely reason is that the large, frequent changes make reuse less helpful, so LPA ends up spending more time repairing the old search than A spends starting over. Even after the startup runs, LPA* still stays clearly slower in the 50×50 high-disturbance case.

5. Discussion & Conclusion

5.1 Takeaway conclusion: guideline / threshold evaluation

The main takeaway from this report is that the comparison between restart replanning with A* and repair replanning with LPA* is not a simple winner-loser result. Instead, the experiment supports a threshold-style conclusion: the better choice depends on both the disturbance level of the map updates and the size of the search space. In the experiment, A* was usually the better choice in the smaller grids and in the high-disturbance cases, where updates were frequent and disruptive. LPA* became more useful in the low-disturbance medium and larger grids, especially after the first run, where reusing earlier search information started to reduce planning time. This means the threshold idea was partly supported "repair planning helps most when changes are limited enough that earlier search results still remain useful", while restart planning stays practical when recomputation is cheap or when changes are too disruptive for reuse to help much.

This result fits the studies used earlier in the paper. Koenig and Likhachev argue that replanning from scratch can be inefficient when most edge costs do not change between replanning episodes, which supports the motivation for incremental repair. Additionally, Honda et al. show that effective replanning depends on the environment and planner setup, which matches the fact that the better method changed across the test cases in this project.

A simple guideline from this report is therefore: use A* when the problem is small or when updates are large and frequent and use LPA* when the search space is larger and changes are limited enough for reuse to matter. This is not a universal rule, but it gives a practical answer to the project goal of building a replanning threshold.

A useful real life way to understand this result is to imagine an autonomous mobile robot in a warehouse or busy restaurant. If only a small part of its route is blocked by a cart or a person, repair-based planning is more attractive because much of the earlier search may remain useful. However, if obstacles are large, frequent, or disruptive enough to keep changing the route, restarting with A* becomes more practical because the value of reuse becomes smaller. This matches earlier work showing that LPA* is most helpful when problems change only slightly and previous search information can be reused, while repair becomes less attractive when changes are more substantial.

From a more modern perspective, Zhu et al. noted on various traditional planners such as A*, Dijkstra, and RRT that these planners perform well in static and well-known environments, but in dynamic or unknown environments they often require repeated replanning, which increases computational overhead and hurts real-time performance (Zhu et al., 2025, p. 2). The review also says that current navigation research is focusing more on real-time adaptation, using multiple types of sensory input, transferring from simulation to the real world, and especially staying safe in changing environments (Zhu et al., 2025, p. 23)

5.2 Limitations

This report has several limitations. First, the experiment is small in scope, it uses a limited set of grid sizes and controlled layouts in an Rviz2-based setting, so the results should be treated as an exploratory study rather than a final rule for all robot navigation problems. Second, change frequency and change size were combined into a single disturbance-level variable to keep the experiment manageable, which means their effects were not fully separated in every test case.

Another limitation is that the comparison is driven mainly by planning time, because path lengths were often the same or very similar for both planners. This makes sense for the threshold question, but it means the report says more about replanning cost than about all possible navigation quality metrices that wasn’t considered.

Finally, this project compares only A* and LPA*, so it should be read as a focused comparison between restart and repair planning rather than a complete study of all replanning methods. Even with these limitations, the report still shows that the restart vs repair choice can be studied experimentally and that the answer depends on the disturbance conditions of the environment.

From a modern perspective, another limitation is that this project does not test several issues that current navigation research now treats as important, including multimodal perception, stronger real time adaptability, and deployment in dynamic environments (Zhu et al., 2025, pp. 1, 23).

References

Honda, K., Yonetani, R., Nishimura, M., & Kozuno, T. (2023). When to replan? An adaptive replanning strategy for autonomous navigation using deep reinforcement learning. *arXiv.* https://doi.org/10.48550/arXiv.2304.12046

Koenig, S., & Likhachev, M. (2005). Fast replanning for navigation in unknown terrain. *IEEE Transactions on Robotics, 21(3)*. https://doi.org/10.1109/TRO.2004.838026

Koenig, S., Likhachev, M., & Furcy, D. (2004). Lifelong planning A*. *Artificial Intelligence, 155(1–2).* https://doi.org/10.1016/j.artint.2003.12.001

Likhachev, M., Ferguson, D. I., Gordon, G. J., Stentz, A., & Thrun, S. (2005). Anytime Dynamic A*: An anytime, replanning algorithm. *In Proceedings of the Fifteenth International Conference on Automated Planning and Scheduling (ICAPS 2005).* https://publications.ri.cmu.edu/anytime-dynamic-a-an-anytime-replanning-algorithm

Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games, 4(2).* https://doi.org/10.1109/TCIAIG.2012.2197681

Sturtevant, N. R., Traish, J., Tulip, J., Uras, T., Koenig, S., Strasser, B., Botea, A., Harabor, D., & Rabin, S. (2015). The grid-based path planning competition: 2014 entries and results. *In Proceedings of the Eighth Annual Symposium on Combinatorial Search (SoCS 2015).* https://research.ibm.com/publications/the-grid-based-path-planning-competition-2014-entries-and-results

Zhu, Y., Wan Hasan, W. Z., Harun Ramli, H. R., Norsahperi, N. M. H., Mohd Kassim, M. S., & Yao, Y. (2025). Deep reinforcement learning of mobile robot navigation in dynamic environment: A review. *Sensors, 25(11).* https://doi.org/10.3390/s25113394